



iLEAPP Module Contributor Guide

leapps.org

June 2026

This guide walks through writing a new iLEAPP artifact module from scratch, wiring it for LAVA output, and submitting it as a pull request. It is written for both Python developers new to iLEAPP and forensic examiners who can code but haven't contributed to the project before.

***Note:** This guide uses iLEAPP as its reference, but the module structure, `__artifacts_v2__` format, helper functions, LAVA integration, and PR process are identical across all LEAPPs tools. Everything here applies equally to ALEAPP, RLEAPP, and VLEAPP. The only differences are the target repository and the artifact paths relevant to each platform.*

What a Module Is

Every artifact iLEAPP parses comes from a module — a single Python file in `scripts/artifacts/`. iLEAPP loads all modules in that folder automatically at runtime. There is no registry to update and no configuration file to edit. Drop a valid module file in the folder and iLEAPP picks it up.

Each module defines what files to look for, how to parse them, and what data to return. The framework handles all output: HTML report, TSV, timeline, and LAVA database.

Prerequisites

- Python 3.10–3.12
- A local clone of the iLEAPP repository: <https://github.com/abrignoni/iLEAPP>
- Dependencies installed: `pip install -r requirements.txt`
- A test extraction or sample image to run against

Module Structure

A module has two required parts: the `__artifacts_v2__` dictionary and the processing function.

1. `__artifacts_v2__`

This must be the first thing in the file. It tells iLEAPP everything it needs to know about the artifact before parsing begins.

```
__artifacts_v2__ = {
    "myArtifact": {
        "name": "My Artifact",
        "description": "Extracts data from MyApp",
        "author": "@YourHandle",
        "creation_date": "2025-01-01",
        "last_update_date": "2025-01-01",
        "requirements": "none",
        "category": "Social Media",
        "notes": "",
        "paths": ('*/mobile/Containers/Data/Application/*/Library/myapp.db',),
        "output_types": "standard",
        "artifact_icon": "message-circle"
    }
}
```

Field reference:

Field	Description
"myArtifact" (key)	Must exactly match the processing function name
name	Display name in reports and LAVA
description	One-line description of what it parses
author	Your name or handle
creation_date	Date you wrote it (YYYY-MM-DD)
last_update_date	Date of last change (YYYY-MM-DD)
requirements	Python package requirements, or "none"
category	Category grouping in reports and LAVA
notes	Research references, caveats, or empty string
paths	Tuple of file path patterns (wildcards supported)
output_types	What output formats to generate (see below)
artifact_icon	Feather icon name for HTML sidebar

output_types values:

Value	Output generated
"standard"	HTML, TSV, timeline, and LAVA — use this for most artifacts
"all"	All of the above plus KML
"lava_only"	LAVA database only, no HTML report
"none"	No output (used for modules that only collect device info)
List e.g. ["html", "lava"]	Specific combination

paths wildcards:

```
# Single file
"paths": ('*/mobile/Library/SMS/sms.db',)

# Multiple patterns
"paths": (
    '*/mobile/Library/CallHistoryDB/CallHistory.storedata',
    '*/mobile/Library/CallHistoryDB/call_history.db',
)

# Match all files in a directory
"paths": ('*/mobile/Library/SMS/Attachments/*',)
```

2. The Processing Function

The function name must match the key in `__artifacts_v2__`. It takes a single context argument and returns a tuple of (data_headers, data_list, source_path).

```
from scripts.ilapfuncs import artifact_processor, get_file_path, get_sqlite_db_records

@artifact_processor
def myArtifact(context):
    files_found = context.get_files_found()
    source_path = get_file_path(files_found, "myapp.db")
    data_list = []

    query = '''
    SELECT timestamp, sender, message
    FROM messages
    '''

    records = get_sqlite_db_records(source_path, query)
    for record in records:
        data_list.append((record[0], record[1], record[2]))

    data_headers = (
        ('Timestamp', 'datetime'),
        'Sender',
        'Message'
    )
    return data_headers, data_list, source_path
```

The `@artifact_processor` decorator handles all output generation. Your function only needs to return clean data.

Returning Data

data_headers

A tuple defining column names. Plain strings for text columns. Tuples of (name, type) for columns that need special rendering.

```
data_headers = (
    ('Timestamp', 'datetime'),          # Rendered as human-readable date/time
    ('Modified Date', 'datetime'),
```

```

'Sender',                # Plain text
'Message',
('Attachment', 'media'), # Rendered as inline image/video in LAVA
('Phone Number', 'phonenumber'), # Formatted phone number
('File Size', 'bytes'),   # Rendered as KB/MB/etc.
)

```

Special column types:

Type	Use for
datetime	Unix epoch integers or Cocoa timestamps converted to UTC
date	Date-only values
media	File paths to images, video, or other media
phonenumber	Phone number strings
bytes	Integer byte counts

data_list

A list of tuples. Each tuple is one row. Values must be in the same order as `data_headers`. Null/missing values should be `None`.

```

data_list = [
    (1681579809, "Alice", "See you tomorrow", "/path/to/image.jpg", "+19195551234", 204800),
    (1681580100, "Bob", "Confirmed", None, "+19195559876", 0),
]

```

source_path

```

# Single file
return data_headers, data_list, source_path

# Multiple source files – add a source column to each row instead
return data_headers, data_list, "See Source File column"

```

Parsing SQLite Databases

Most iOS artifacts are in SQLite databases. Use `get_sqlite_db_records` from `ilapfuncs`.

```

from scripts.ilapfuncs import artifact_processor, get_file_path, get_sqlite_db_records

@artifact_processor
def myArtifact(context):
    files_found = context.get_files_found()
    source_path = get_file_path(files_found, "myapp.db")
    data_list = []

    query = '''
    SELECT
        ZMESSAGE_DATE,
        ZSENDER,
        ZBODY
    FROM ZMESSAGE

```

```

ORDER BY ZMESSAGE_DATE
'''

records = get_sqlite_db_records(source_path, query)
for record in records:
    data_list.append((record[0], record[1], record[2]))

data_headers = (
    ('Timestamp', 'datetime'),
    'Sender',
    'Message'
)
return data_headers, data_list, source_path

```

Parsing Plist Files

from scripts.ilapfuncs import artifact_processor, get_file_path, get_plist_file_content, convert_plist_date

```

@artifact_processor
def myArtifact(context):
    files_found = context.get_files_found()
    source_path = get_file_path(files_found, "com.example.myapp.plist")
    data_list = []

    pl = get_plist_file_content(source_path)
    if not pl or not isinstance(pl, dict):
        return (), [], ''

    for entry in pl.get('entries', []):
        timestamp = convert_plist_date_to_utc(entry.get('date', ''))
        value = entry.get('value', '')
        data_list.append((timestamp, value))

    data_headers = (
        ('Date', 'datetime'),
        'Value'
    )
    return data_headers, data_list, source_path

```

Timestamp Conversion

Always store timestamps as UTC. Use the appropriate conversion helper.

```

from scripts.ilapfuncs import (
    convert_cocoa_core_data_ts_to_utc, # Cocoa/Core Data epoch (seconds since 2001-01-01)
    convert_unix_ts_to_utc,            # Unix epoch in seconds
    convert_plist_date_to_utc,         # Plist datetime objects
    convert_ts_human_to_utc,           # Human-readable timestamp strings
)

```

Adding LAVA Conversation View

If your artifact contains messages, add `data_views` to the `__artifacts_v2__` definition to enable conversation view in LAVA.

```
__artifacts_v2__ = {
    "myMessaging": {
        ...
        "data_views": {
            "conversation": {
                "conversationDiscriminatorColumn": "Thread ID",
                "conversationLabelColumn": "Contact Name",
                "textColumn": "Message",
                "directionColumn": "Direction",
                "directionSentValue": "Sent",
                "timeColumn": "Timestamp",
                "senderColumn": "Sender",
                "mediaColumn": "Attachment"
            }
        }
    }
}
```

The column names in `data_views` must match the display names defined in `data_headers` — not the raw SQL column names.

Conversation view fields:

Field	Required	Description
<code>conversationDiscriminatorColumn</code>	yes	Column that groups messages into threads
<code>conversationLabelColumn</code>	no	Column used to label threads in the sidebar
<code>textColumn</code>	yes	Column containing message text
<code>directionColumn</code>	yes	Column indicating sent vs received
<code>directionSentValue</code>	yes	Value in <code>directionColumn</code> that means sent (string or integer)
<code>timeColumn</code>	yes	Column containing message timestamp
<code>senderColumn</code>	no	Column containing sender identity for received messages
<code>sendMessageLabelColumn</code>	no	Column to use as sender label for sent messages
<code>sendMessageStaticLabel</code>	no	Fixed string label for sent messages (e.g. "This Phone")
<code>mediaColumn</code>	no	Column containing media attachment reference

Complete Module Example

A full working module for a hypothetical SQLite-based messaging artifact.

```
__artifacts_v2__ = {
    "exampleMessages": {
        "name": "Example App - Messages",
        "description": "Extracts messages from Example App",
        "author": "@YourHandle",
    }
}
```

```

        "creation_date": "2025-01-01",
        "last_update_date": "2025-01-01",
        "requirements": "none",
        "category": "Messaging",
        "notes": "",
        "paths": ('*/mobile/Containers/Data/Application/*/Library/Application Support/example.db',),
        "output_types": "standard",
        "artifact_icon": "message-circle",
        "data_views": {
            "conversation": {
                "conversationDiscriminatorColumn": "Thread ID",
                "conversationLabelColumn": "Contact",
                "textColumn": "Message",
                "directionColumn": "Direction",
                "directionSentValue": "Sent",
                "timeColumn": "Timestamp",
                "senderColumn": "Contact",
                "mediaColumn": "Attachment"
            }
        }
    }
}

from scripts.ilapfuncs import (
    artifact_processor,
    get_file_path,
    get_sqlite_db_records,
    convert_unix_ts_to_utc
)

@artifact_processor
def exampleMessages(context):
    files_found = context.get_files_found()
    source_path = get_file_path(files_found, "example.db")
    data_list = []

    query = '''
    SELECT
        m.timestamp,
        m.thread_id,
        c.display_name,
        CASE m.direction WHEN 1 THEN 'Sent' ELSE 'Received' END,
        m.body,
        m.attachment_path
    FROM messages m
    LEFT JOIN contacts c ON m.contact_id = c.id
    ORDER BY m.timestamp
    '''

    records = get_sqlite_db_records(source_path, query)
    for record in records:

```

```

        timestamp = convert_unix_ts_to_utc(record[0])
        data_list.append((
            timestamp,
            record[1], # thread_id
            record[2], # contact name
            record[3], # direction
            record[4], # message body
            record[5], # attachment path
        ))

    data_headers = (
        ('Timestamp', 'datetime'),
        ('Thread ID',
         'Contact',
         'Direction',
         'Message',
         ('Attachment', 'media')),
    )
    return data_headers, data_list, source_path

```

Testing Your Module

Run iLEAPP against a test extraction and verify your artifact appears in the output.

CLI:

```
python ileapp.py -t fs -i /path/to/extraction -o /path/to/output
```

GUI:

```
python ileappGUI.py
```

Check the output folder for:

- An HTML report file for your artifact
- A TSV file with the same data
- The `_lava_artifacts.db` containing your artifact's table
- The `_lava_data.lava` containing your artifact's definition

Verify the LAVA output by opening the project in LAVA and confirming your artifact appears in the correct category with the correct columns.

Submitting a Pull Request

1. **Fork** the iLEAPP repository on GitHub.
2. **Create a branch** named for your artifact: `git checkout -b artifact/my-app-messages`
3. **Place your module** in `scripts/artifacts/myModule.py`
4. **Test** against a real extraction. Confirm data appears correctly in both the HTML report and LAVA.
5. **Commit** with a clear message: `Add MyApp message parser`
6. **Open a pull request** against the main branch of `abrignoni/iLEAPP`

PR checklist:

- `__artifacts_v2__` is the first thing in the file
- Function name matches the `__artifacts_v2__` key exactly
- All timestamps are converted to UTC
- `data_headers` types are set correctly for datetime, media, and phonenumbers columns
- `output_types` is set to "standard" unless there is a specific reason otherwise
- Module tested against a real extraction
- Conversation `data_views` added if the artifact contains messages
- Author field populated in `__artifacts_v2__`

Key Helper Functions Reference

All helpers are in `scripts/ilapfuncs.py`.

Function	Use
<code>get_file_path(files_found, filename)</code>	Get path to a specific file from the matched files list
<code>get_sqlite_db_records(path, query)</code>	Execute a query and return rows
<code>get_plist_file_content(file_path)</code>	Read and parse a plist file
<code>convert_unix_ts_to_utc(ts)</code>	Unix epoch to UTC string
<code>convert_cocoa_core_data_ts_to_utc(ts)</code>	Cocoa epoch (2001-01-01) to UTC string
<code>convert_plist_date_to_utc(ts)</code>	Plist date object to UTC string
<code>convert_ts_human_to_utc(ts)</code>	Human-readable timestamp string to UTC
<code>logfunc(message)</code>	Write a message to the iLEAPP log

leapps.org